

CO1
ASSIGNMENT - 1
Visualizing Data

Department	AIDS
Course Code / Course Name	DSA0601 / Data Handling and Visualization
Experiment	Data Visualization using R
Name	DEEKSHITHA U
Register Number	192424055
Date	06 October 2026
Day	Tuesday

Q1. Visual to R Code: Aesthetics and a Log Axis

Given Data

Name	Literacy_pct	Population_K	Zone
Karur	55	2000	East
Vellore	70	500	South
Theni	90	100	North
Salem	95	200	West
Erode	80	10000	North
Tirupur	55	20	East
Trichy	75	50000	South

The given graph is a **scatter plot** in which:

- X-axis → Literacy_pct
- Y-axis → Population_K
- Colour → Zone
- Y-axis uses a **logarithmic scale**

(a) R Program

```
# Load ggplot2
library(ggplot2)
# -----
# Step 1: Create the data frame
# -----
df <- data.frame(
  Name = c(
    "Karur",
    "Vellore",
    "Theni",
    "Salem",
    "Erode",
    "Tirupur",
    "Trichy"
  ),
  Literacy_pct = c(
    55, 70, 90, 95, 80, 55, 75
  ),
  Population_K = c(
    2000, 500, 100, 200, 10000, 20, 50000
  ),
  Zone = c(
    "East",
    "South",
    "North",
    "West",
    "North",
    "East",
    "South"
  )
)
# -----
# Step 2: Create the scatter plot
```

```
# -----
ggplot(
  df,
  aes(
    x = Literacy_pct,
    y = Population_K,
    color = Zone
  )
) +
  geom_point(size = 4) +
  # Logarithmic transformation of Y-axis
  scale_y_log10() +
  labs(
    title = "Literacy Percentage vs Population",
    x = "Literacy_pct",
    y = "Population_K (log scale)",
    color = "Zone"
  ) +

  theme_minimal()
```

Explanation of the program

The `data.frame()` function creates the dataset containing the four variables.

```
ggplot(df, aes(
  x = Literacy_pct,
  y = Population_K,
  color = Zone
))
```

maps the variables to visual properties.

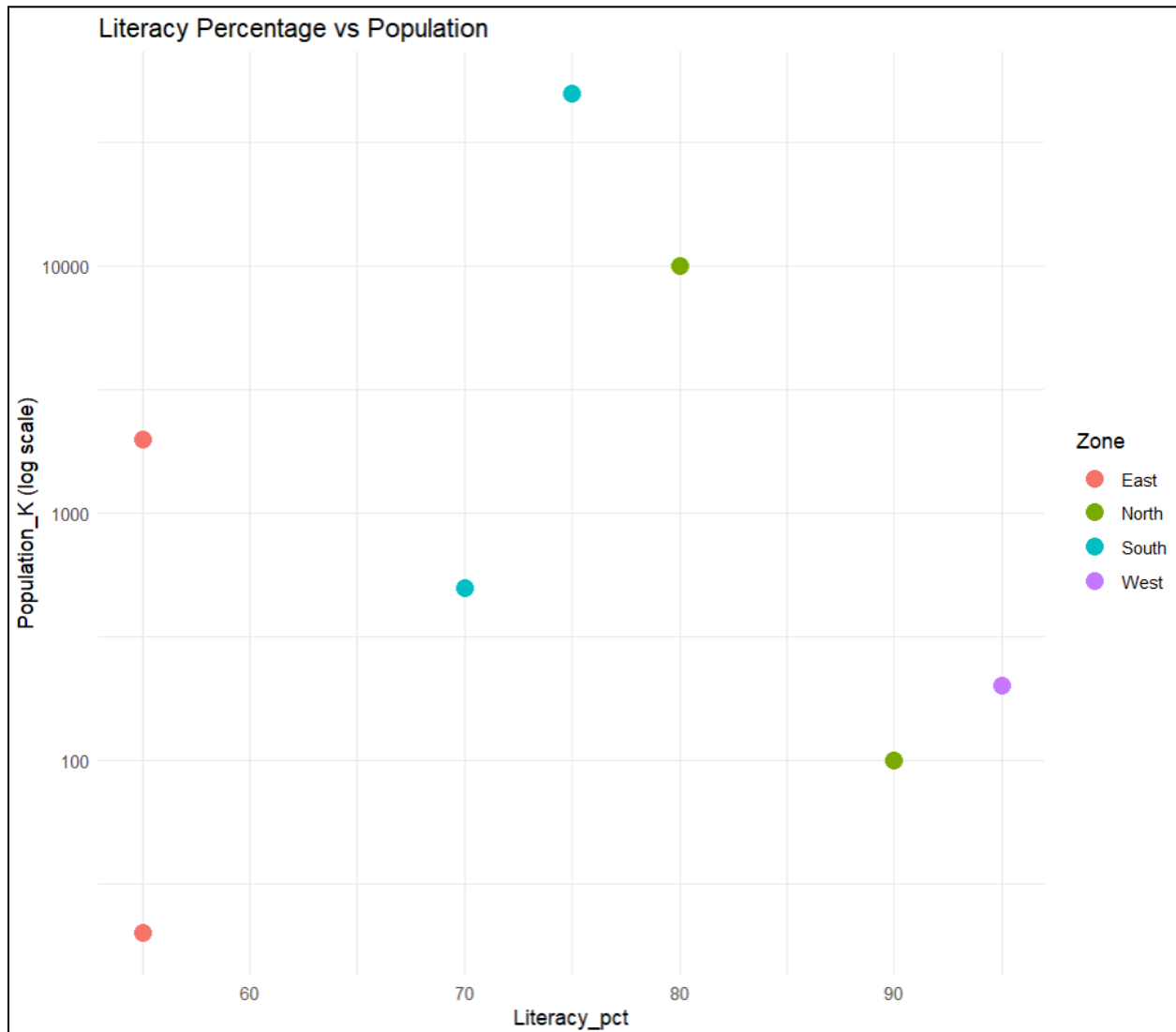
`geom_point()` creates the scatter plot.

```
scale_y_log10()
```

converts the Population axis into a **logarithmic scale**.

Finally, `labs()` provides appropriate axis labels and a legend title.

Output



(b) Aesthetic Used for Each Variable and Why Population_K Needs a Nonlinear Axis

1. **Literacy_pct** → X-axis

x = Literacy_pct

Literacy_pct is a numerical continuous variable, so it is represented using the **horizontal position**.

The X-axis shows literacy values ranging from **55% to 95%**.

2. **Population_K** → Y-axis

y = Population_K

Population_K is also a numerical continuous variable, so it is represented using the **vertical position**.

However, its values range from:

20 to 50,000

This is a very large range.

3. Zone → Colour

color = Zone

Zone is a categorical variable containing:

- East
- West
- North
- South

Therefore, colour is an appropriate aesthetic because it allows the different zones to be distinguished visually.

Why does Population_K need a nonlinear/logarithmic axis?

The population values are highly spread out:

20
100
200
500
2,000
10,000
50,000

The maximum value, **50,000**, is much larger than the minimum value, **20**.

If a normal linear scale were used, the smaller values such as 20, 100 and 200 would be compressed close to the bottom of the graph, making them difficult to distinguish.

Therefore, a logarithmic transformation is used:

scale_y_log10()

A logarithmic scale makes values across different orders of magnitude easier to visualize.

Interpretation of the Graph

The graph shows the relationship between **literacy percentage and population** for the seven locations.

Some important observations are:

- **Trichy** has the highest population, with **50,000**, and a literacy percentage of **75%**.
- **Erode** has a population of **10,000** and literacy of **80%**.
- **Karur** has a population of **2,000** and literacy of **55%**.
- **Tirupur** has the lowest population, **20**, with literacy of **55%**.
- **Salem** has the highest literacy percentage, **95%**, with population **200**.
- The points are separated by Zone using different colours.

The graph does not show a simple direct relationship between literacy and population. For example, Trichy has very high population but only 75% literacy, whereas Salem has 95% literacy but a much smaller population.

Q2. Visual to Table: Reading a Colour Scale

The heat map represents **Sales Score** for 4 regions across 5 quarters.

The legend contains five colour classes:

Colour Class	Sales Score
Lightest	0–20
	20–40
	40–60
	60–80
Darkest	80–100

The darker the colour, the higher the sales score.

R Code

```
library(ggplot2)
# Create the sales score data
sales_data <- data.frame(
  Region = c(
    "North", "North", "North", "North", "North",
    "South", "South", "South", "South", "South",
    "East", "East", "East", "East", "East",
    "West", "West", "West", "West", "West"
  ),
  Quarter = c(
    "Q1", "Q2", "Q3", "Q4", "Q5",
    "Q1", "Q2", "Q3", "Q4", "Q5",
    "Q1", "Q2", "Q3", "Q4", "Q5",
    "Q1", "Q2", "Q3", "Q4", "Q5"
  ),
  Score = c(
    70, 50, 90, 70, 90,
    10, 90, 10, 50, 50,
    70, 70, 70, 70, 10,
    10, 70, 30, 30, 30
  )
)
```

```

)
)
# Convert variables into factors
sales_data$Region <- factor(
  sales_data$Region,
  levels = c("North", "South", "East", "West")
)

sales_data$Quarter <- factor(
  sales_data$Quarter,
  levels = c("Q1", "Q2", "Q3", "Q4", "Q5")
)

# Create heat map
ggplot(sales_data,
  aes(x = Quarter, y = Region, fill = Score)) +

  geom_tile(color = "white", linewidth = 1) +

  scale_fill_gradient(
    low = "lightblue",
    high = "darkblue",
    breaks = c(10, 30, 50, 70, 90),
    labels = c("0-20", "20-40", "40-60", "60-80", "80-100"),
    name = "Sales score"
  ) +

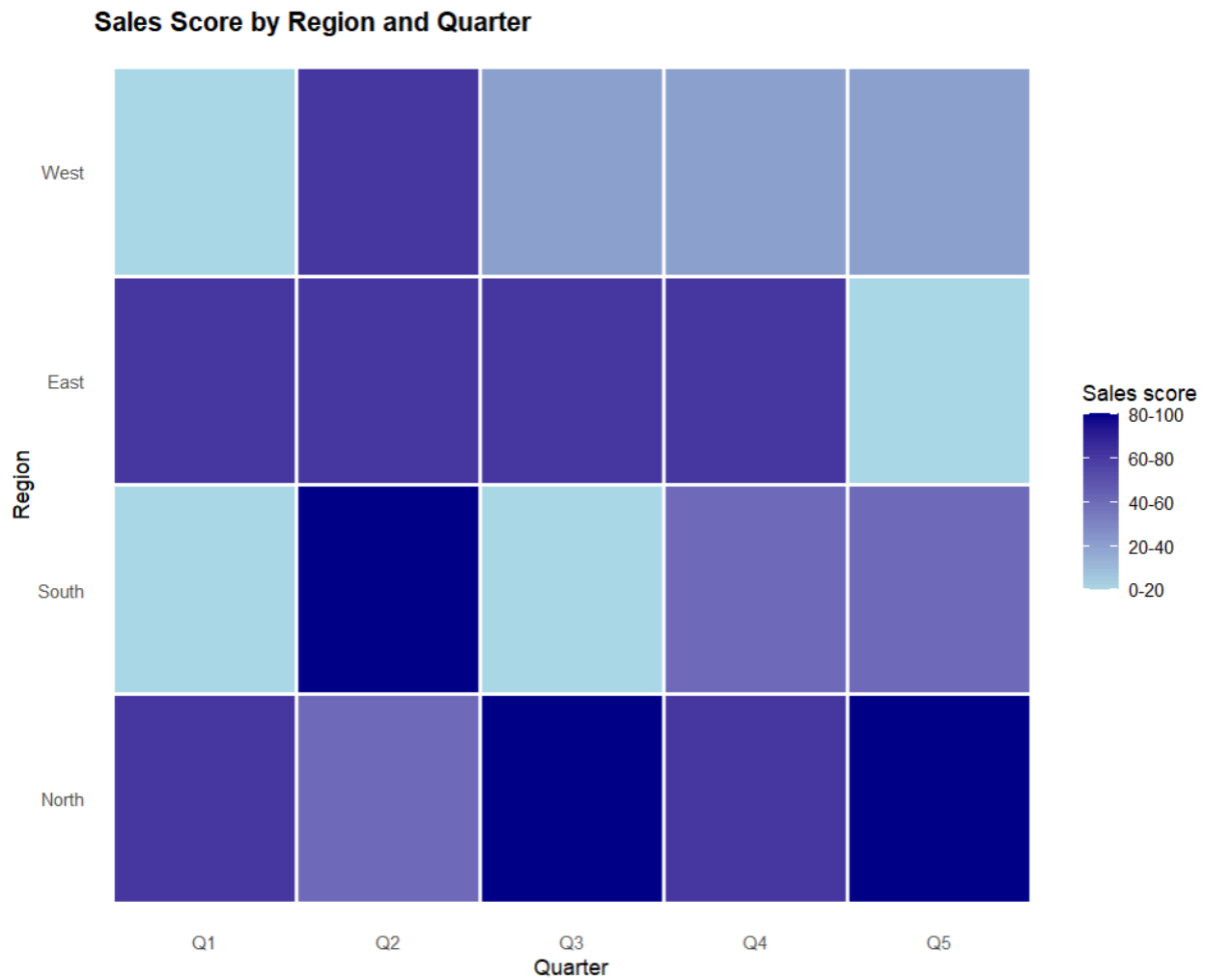
  labs(
    title = "Sales Score by Region and Quarter",
    x = "Quarter",
    y = "Region"
  ) +

  theme_minimal() +

  theme(
    plot.title = element_text(face = "bold"),
    panel.grid = element_blank()
  )

```

Output



(a) Reconstruct the 4×5 Table

By reading the colour of each cell from the heatmap and matching it with the legend, the table can be reconstructed as follows:

Region	Q1	Q2	Q3	Q4	Q5
North	60–80	40–60	80–100	60–80	80–100
South	0–20	80–100	0–20	40–60	40–60
East	60–80	60–80	60–80	60–80	0–20
West	0–20	60–80	20–40	20–40	20–40

Therefore, the reconstructed table is:

	Q1	Q2	Q3	Q4	Q5
North	60–80	40–60	80–100	60–80	80–100
South	0–20	80–100	0–20	40–60	40–60
East	60–80	60–80	60–80	60–80	0–20
West	0–20	60–80	20–40	20–40	20–40

(b) Name the Type of Colour Scale and Justify It

The colour scale used in the heat map is a:

Sequential colour scale

Justification

A sequential colour scale is used when the data represents values that progress from **low to high**.

In this heat map:

- 0–20 → lightest colour
- 20–40 → slightly darker
- 40–60 → medium colour
- 60–80 → darker colour
- 80–100 → darkest colour

There is a clear ordered progression from **low sales scores to high sales scores**.

Therefore, the scale is sequential rather than qualitative or diverging.

Why it is not qualitative

A qualitative colour scale is normally used for categories that have no natural numerical order, such as:

- North
- South
- East
- West

But the heatmap's colours represent **ordered numerical sales scores**, so a sequential scale is more appropriate.

Why it is not diverging

A diverging scale is normally used when there is a meaningful central value, such as:

- Negative → zero → positive
- Below target → target → above target

There is no such central reference point here. The sales scores simply increase from 0 to 100.

Therefore, **sequential is the correct answer**.

(c) Cells in the Highest Class and Why "Darker = Higher" Is Intuitive

The highest class is:

80–100

The cells belonging to this class are:

1. **North – Q3**
2. **North – Q5**
3. **South – Q2**

Why is darker = higher intuitive?

Darker shades generally appear visually stronger or more prominent than lighter shades.

Therefore, when a sequential colour scale progresses from light to dark, viewers can naturally interpret:

Light → Low value

Dark → High value

This allows high-sales cells to be identified quickly without reading every numerical value.

Reconstructed table

Region	Q1	Q2	Q3	Q4	Q5
North	60–80	40–60	80–100	60–80	80–100
South	0–20	80–100	0–20	40–60	40–60
East	60–80	60–80	60–80	60–80	0–20
West	0–20	60–80	20–40	20–40	20–40

Q3. Table to Visual: Mapping Data onto Aesthetics

Given data

ID	RAM_GB	Battery_hrs	Brand	Price_K
M1	10	16	BrandC	20
M2	2	12	BrandC	40
M3	10	10	BrandB	10
M4	7	12	BrandB	30
M5	7	20	BrandB	30
M6	2	14	BrandC	30
M7	11	16	BrandA	40
M8	3	10	BrandC	10

(a) Data types

Variable	Data Type	Reason
RAM_GB	Continuous	RAM is a numerical measurement and can theoretically take fractional values.
Battery_hrs	Continuous	Battery life is a numerical measurement of time.
Brand	Nominal	Brands are categories with no natural order.
Price_K	Continuous	Price is a numerical quantity representing monetary value.

Answer:

RAM_GB – Continuous

Battery_hrs – Continuous

Brand – Nominal

Price_K – Continuous

(b) Suitable aesthetics and scale type

A suitable visualization is a **bubble scatter plot**.

Variable	Aesthetic	Scale Type
RAM_GB	X-axis	Continuous
Battery_hrs	Y-axis	Continuous
Brand	Color	Discrete / Nominal
Price_K	Size of point	Continuous

Therefore:

RAM_GB → X position

Battery_hrs → Y position

Brand → Color

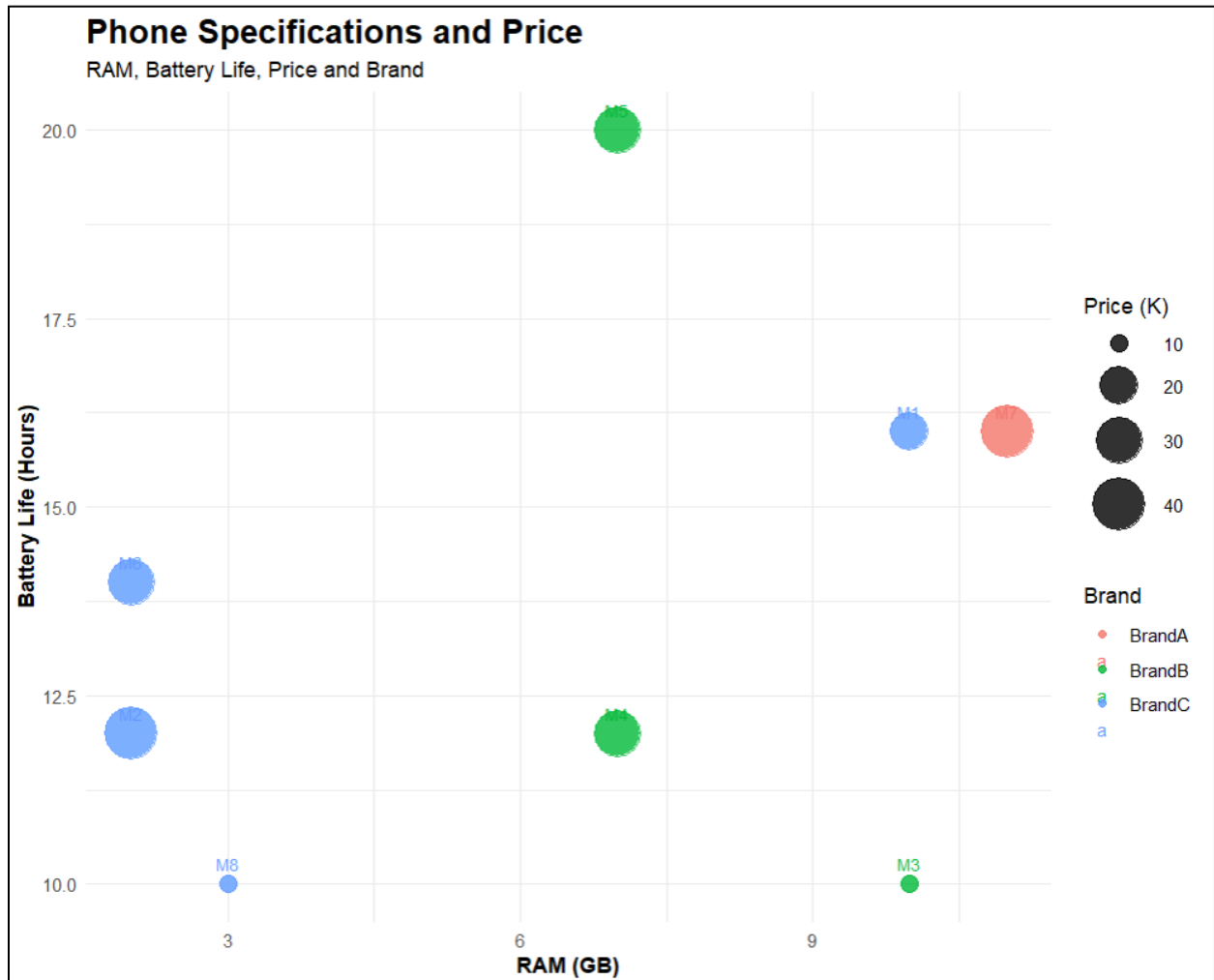
Price_K → Point size

Why this is suitable

- RAM and Battery are numerical variables, so they are naturally represented using the **X and Y axes**.
- Brand is categorical, so **color** is appropriate.
- Price is numerical, so **size** can represent its magnitude.

(c) Sketch the chart

The appropriate chart is a **bubble scatter plot**.



Important: In the actual assignment, draw the scatter/bubble plot neatly and show:

- X-axis = RAM_GB
- Y-axis = Battery_hrs
- Brand color legend
- Price_K size legend

(d) Why would mapping Price_K to shape be a poor choice?

Mapping **Price_K to shape** is a poor choice because **Price_K is a continuous numerical variable**.

There are only a limited number of easily distinguishable shapes. It is difficult for a viewer to understand numerical differences such as:

10K → 20K → 30K → 40K

using shapes.

Instead, **size** is more suitable because larger prices can be represented by larger bubbles, making the numerical differences easier to compare.

Final answer:

Price_K is a continuous variable, whereas shape is better suited for categorical variables. There are only a few easily distinguishable shapes, so using shape would make numerical differences in price difficult to perceive. Therefore, mapping **Price_K** to point size is a better choice.

Q4. R Code to Visual: Polar Coordinates

The question specifically says “**Do not run it.**” We therefore interpret the given R code.

The important part is:

```
ggplot(df, aes(x = Time_slot, y = Web_Visits_K)) +  
  geom_col(fill = "steelblue", width = 1) +  
  coord_polar() +  
  scale_y_continuous(limits = c(0, 100))
```

The data is:

Time slot	Web Visits (K)
0h	100
3h	60
6h	40
9h	100
12h	50
15h	30
18h	100
21h	80

(a) Sketch the chart. Show the angle labels and radial scale.

Because `coord_polar()` is used, the ordinary bar chart is converted into a **polar/radial bar chart**.

There are **8 time slots**, so the circle is divided into 8 equal sections.

Each section has an angle of:

$$\frac{360^\circ}{8} = 45^\circ$$

Therefore:

Time	Angle
0h	0°
3h	45°
6h	90°
9h	135°
12h	180°
15h	225°
18h	270°
21h	315°

The radial scale goes from:

0 → 20 → 40 → 60 → 80 → 100

For the actual exam/assignment, draw a **proper circular polar bar chart** with:

- 0h, 3h, 6h, 9h, 12h, 15h, 18h, 21h around the circle.
- Radial values 0, 20, 40, 60, 80, 100.
- Bar lengths corresponding to the given values.

Important observation

The bars for:

- 0h = 100
- 9h = 100
- 18h = 100

will reach the maximum radius.

The shortest bar is:

- 15h = 30

(b) What do angle and radius encode? Name the coordinate system.

Angle

The **angle** represents the **Time_slot**.

The eight time slots are distributed equally around the circle:

Time_slot → Angle

Each time interval occupies:

45°

Radius

The **radius** represents **Web_Visits_K**.

For example:

30K → radius = 30

60K → radius = 60

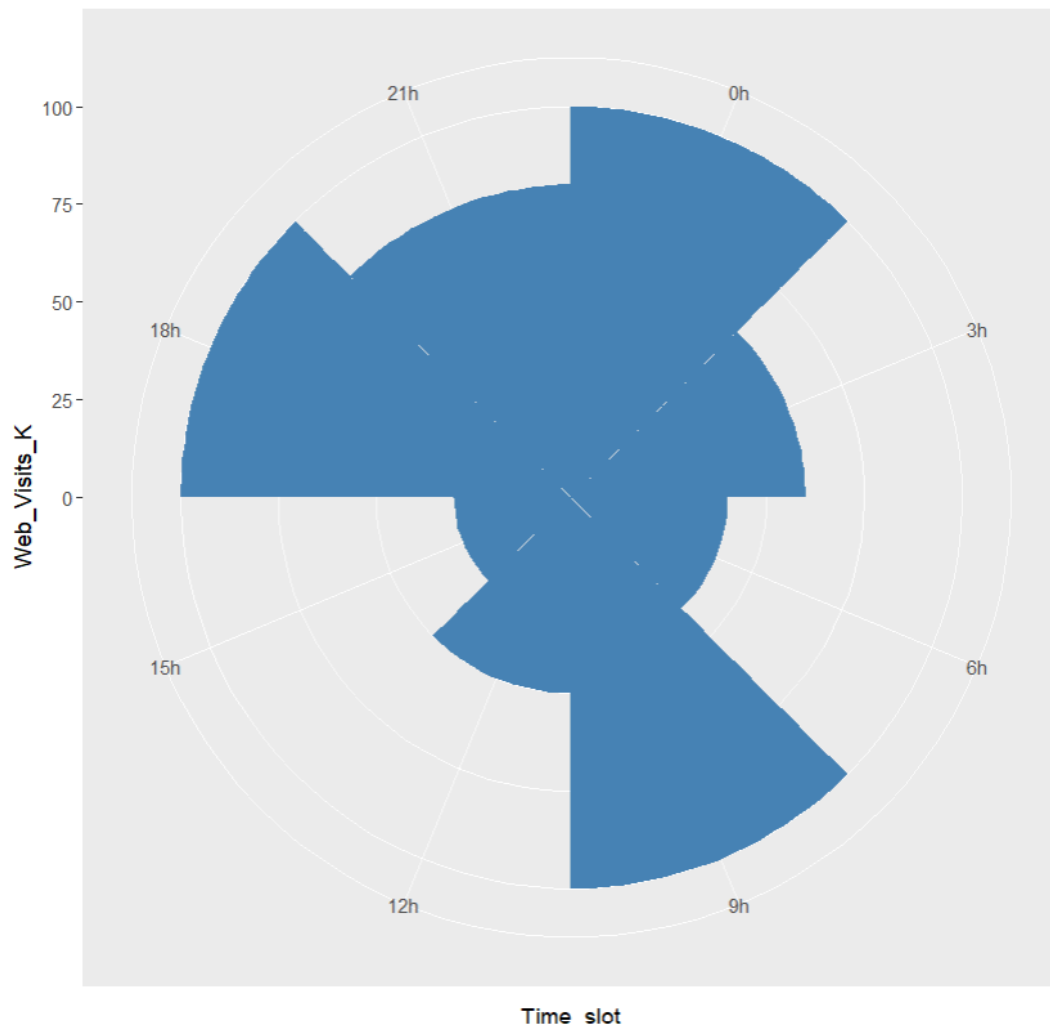
100K → radius = 100

Coordinate system

The graph uses a **polar coordinate system** because the code contains:

coord_polar()

Answer:



(c) Which change gives the same data as an ordinary Cartesian bar chart?

Simply remove the **coord_polar()** line.

Original:

```
ggplot(df, aes(x = Time_slot, y = Web_Visits_K)) +  
  geom_col(fill = "steelblue", width = 1) +  
  coord_polar() +  
  scale_y_continuous(limits = c(0, 100))
```

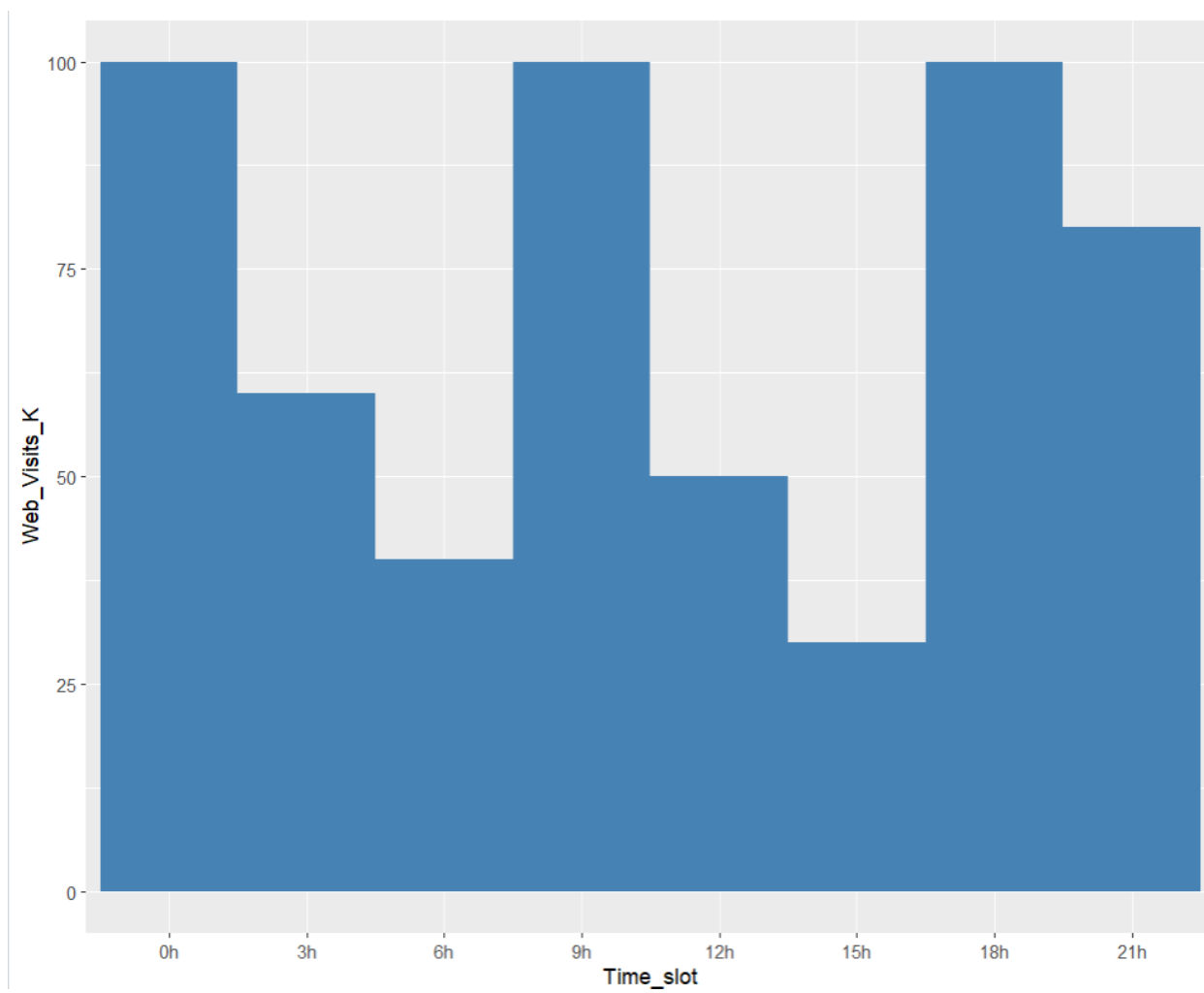
Modified code:

```
ggplot(df, aes(x = Time_slot, y = Web_Visits_K)) +  
  geom_col(fill = "steelblue", width = 1) +
```

`scale_y_continuous(limits = c(0, 100))`

This produces an **ordinary Cartesian bar chart**.

Final answer:



Q5. Design Decision: Choosing Coordinate Systems and Axes

The question asks us to visualize **two different datasets**, so the coordinate system and axis transformation should be chosen according to the nature of each dataset.

(a) Choose a coordinate system for each dataset and justify your choice.

Dataset 1: Monthly rainfall over a year

Coordinate system: Cartesian coordinate system

Use:

- X-axis → Months
- Y-axis → Rainfall

A **line chart** is suitable because the data represents rainfall measured over time.

Justification

The Cartesian coordinate system is appropriate because:

1. Months have a natural chronological order.
2. The X-axis can represent the 12 months.
3. The Y-axis can represent rainfall amount.
4. A line chart clearly shows the increase and decrease in rainfall throughout the year.

Therefore:

Dataset 1 → Cartesian coordinates → Line chart

Dataset 2: Annual revenues of 30 companies

Coordinate system: Cartesian coordinate system

Use:

- X-axis → Companies
- Y-axis → Annual Revenue

A **bar chart** or point plot can be used to compare the revenues.

However, because the revenues range from **₹1 crore to ₹50,000 crore**, the data is highly skewed. Therefore, a **logarithmic Y-axis** should be used.

Justification

1. Revenue is a numerical continuous variable.
2. The values span several orders of magnitude.
3. A normal linear scale would make small-revenue companies difficult to distinguish.
4. A logarithmic scale compresses the very large values and makes smaller values more visible.

Therefore:

Dataset 2 → Cartesian coordinates + logarithmic Y-axis → Bar/point chart

(b) Axis transformation for Dataset 2

The appropriate transformation is a:

Logarithmic transformation

In `ggplot2`, we can use:

`scale_y_log10()`

Why is it useful?

The revenue ranges from:

₹1crore to ₹50,000crore

This is a very large range.

With a normal linear scale, companies with low revenue would be compressed near zero.

A logarithmic scale transforms the values approximately according to their powers of 10:

1 → 10^0

10 → 10^1

100 → 10^2

1,000 $\rightarrow 10^3$

10,000 $\rightarrow 10^4$

50,000 $\rightarrow 5 \times 10^4$

Thus, the logarithmic scale:

- Compresses extremely large revenue values.
- Makes smaller companies easier to compare.
- Allows differences across several orders of magnitude to be visualized clearly.
- Improves the readability of the graph.

Answer:

Dataset 2 should use a logarithmic Y-axis (`scale_y_log10()`) because annual revenues vary from ₹1 crore to ₹50,000 crore. The log transformation reduces the dominance of very large values and makes the smaller values more visible.

(c) `ggplot2` function calls for both choices

Dataset 1 — Monthly Rainfall

Example data:

```
library(ggplot2)

rainfall <- data.frame(

  Month = c("Jan", "Feb", "Mar", "Apr", "May", "Jun",
            "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"),

  Rainfall = c(20, 15, 30, 45, 80, 100,
               120, 110, 90, 70, 40, 25)

)

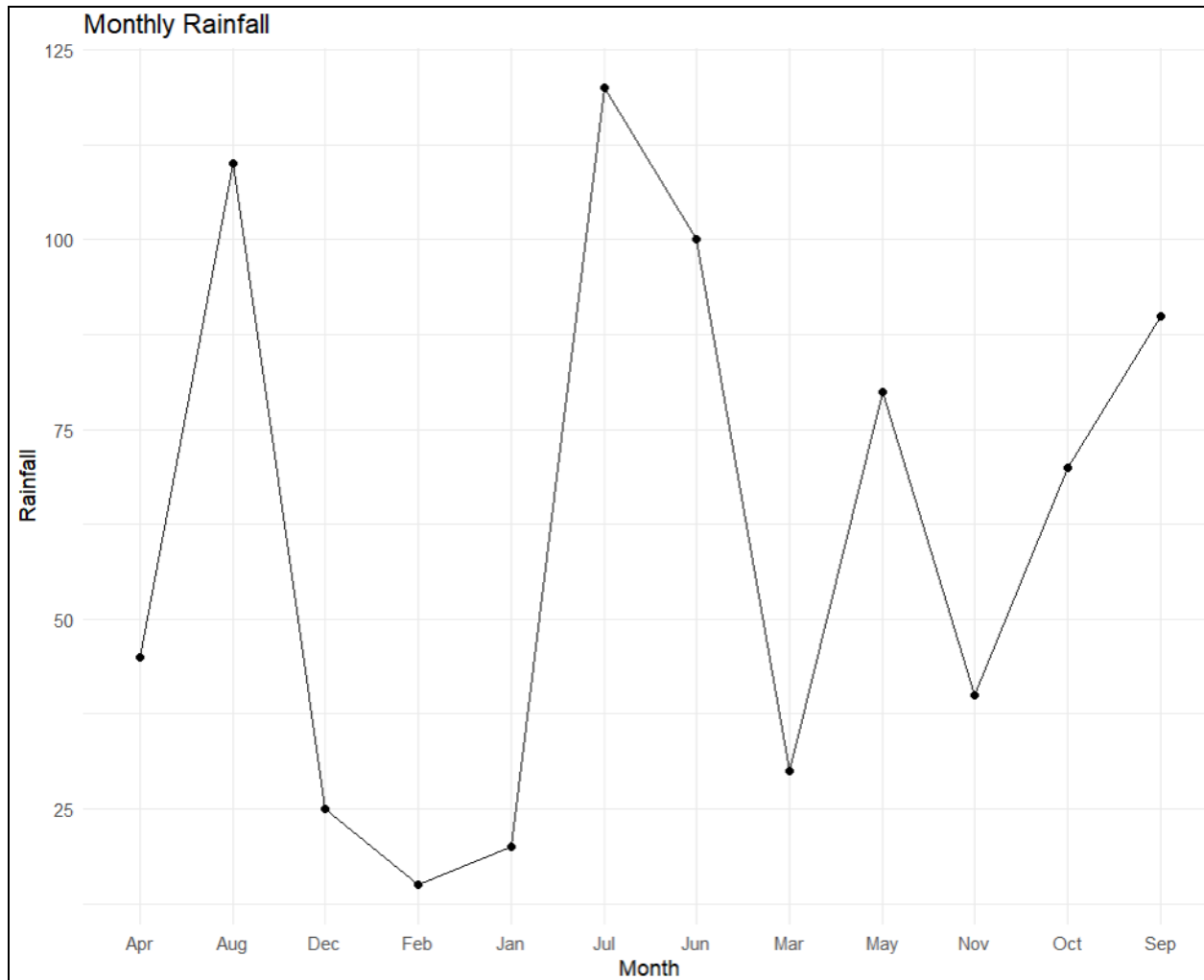
ggplot(rainfall, aes(x = Month, y = Rainfall, group = 1)) +
  geom_line() +
```

```
geom_point() +  
labs(  
  title = "Monthly Rainfall",  
  x = "Month",  
  y = "Rainfall"  
) +  
theme_minimal()
```

Here:

```
aes(x = Month, y = Rainfall)
```

uses a **Cartesian coordinate system by default**.



Dataset 2 — Company Revenue

Example data:

```
revenue <- data.frame(  
  Company = paste0("Company_", 1:30),  
  Revenue = c(  
    1, 5, 10, 20, 35, 50, 80, 120, 200, 300,  
    450, 600, 800, 1000, 1500, 2000, 2500, 3000,  
    4000, 5000, 6500, 8000, 10000, 12000, 15000,  
    18000, 22000, 30000, 40000, 50000
```

```

)
)
ggplot(revenue, aes(x = Company, y = Revenue)) +
  geom_col() +
  scale_y_log10() +
  labs(
    title = "Annual Revenue of Companies",
    x = "Company",
    y = "Annual Revenue (₹ Crore, Log Scale)"
  ) +
  theme_minimal() +
  theme(
    axis.text.x = element_text(angle = 90, hjust = 1)
  )

```

The important function call is:

```
scale_y_log10()
```

which applies the logarithmic transformation to the Y-axis.

